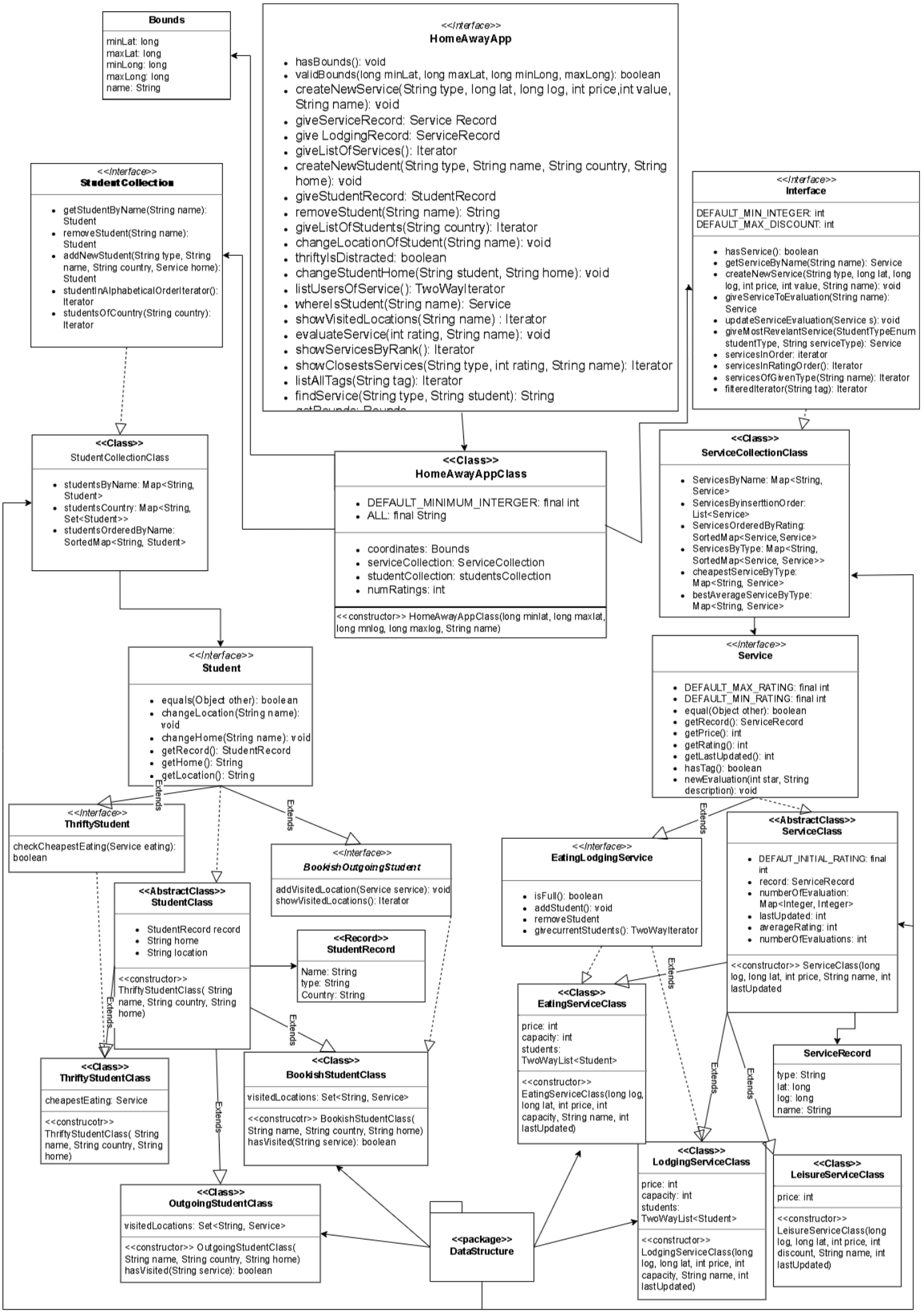
FACULDADE DE CIÊNCIAS E TECNOLOGIA

Algoritmos e Estruturas De Dados(LEI) - 2025/2026 - 1ºSemestre

Relatório Final do 2º Projeto - *HomeAway From Home!*

André Martins Nº 70920

Ricardo Moreira Nº 71283


Bounds
minLat: long
maxLat: long
minLong: long
maxLong: long
name: String
<<Interface>>
HomeAwayApp
hasBounds(): void
validBounds(long minLat, long maxLat, long minLong, maxLong): boolean
createNewService(String type, long lat, long log, int price,int value, String name): void
giveServiceRecord: Service Record
give LodgingRecord: ServiceRecord
giveListOfServices(): Iterator
createNewStudent(String type, String name, String country, String home): void
giveStudentRecord: StudentRecord
removeStudent(String name): String
giveListOfStudents(String country): Iterator
changeLocationOfStudent(String name): void
thriftyIsDistracted: boolean
changeStudentHome(String student, String home): void
listUsersOfService(): TwoWayIterator
wherelsStudent(String name): Service
showVisitedLocations(String name) : Iterator
evaluateService(int rating, String name): void
showServicesByRank(): Iterator
showClosestsServices(String type, int rating, String name): Iterator
listAllTags(String tag): Iterator
findService(String type, String student): String
<<Interface>>
StudentCollection
getStudentByName(String name): Student
removeStudent(String name): Student
addNewStudent(String type, String name, String country, Service home): Student
studentInAlphabeticalOrderIterator(): Iterator
studentsOfCountry(String country): Iterator
<<Interface>>
Interface
DEFAULT_MIN_INTEGER: int
DEFAULT_MAX_DISCOUNT: int
hasService(): boolean
getServiceByName(String name): Service
createNewService(String type, long lat, long log, int price, int value, String name): void
giveServiceToEvaluation(String name): Service
updateServiceEvaluation(Service s): void
giveMostRevelantService(StudentTypeEnum studentType, String serviceType): Service
servicesInOrder: iterator
servicesInRatingOrder(): Iterator
servicesOfGivenType(String name): Iterator
filteredIterator(String tag): Iterator
<<Class>>
StudentCollectionClass
studentsByName: Map<String, Student>
studentsCountry: Map<String, Set<Student>>
studentsOrderedByName: SortedMap<String, Student>
<<Class>>
HomeAwayAppClass
DEFAULT_MINIMUM_INTERGER: final int
ALL: final String
coordinates: Bounds
serviceCollection: ServiceCollection
studentCollection: studentsCollection
numRatings: int
<<constructor>> HomeAwayAppClass(long minlat, long maxlat, long mnlog, long maxlog, String name)
<<Class>>
ServiceCollectionClass
ServicesByName: Map<String, Service>
ServicesByinserttionOrder: List<Service>
ServicesOrderedByRating: SortedMap<Service,Service>
ServicesByType: Map<String, SortedMap<Service, Service>>
cheapestServiceByType: Map<String, Service>
bestAverageServiceByType: Map<String, Service>
<<Interface>>
Student
equals(Object other): boolean
changeLocation(String name): void
changeHome(String name): void
getRecord(): StudentRecord
getHome(): String
getLocation(): String
<<Interface>>
Service
DEFAULT_MAX_RATING: final int
DEFAULT_MIN_RATING: final int
equal(Object other): boolean
getRecord(): ServiceRecord
getPrice(): int
getRating(): int
getLastUpdated(): int
hasTag(): boolean
newEvaluation(int star, String description): void
Extends
<<Interface>>
ThriftyStudent
checkCheapestEating(Service eating): boolean
<<Interface>>
BookishOutgoingStudent
addVisitedLocation(Service service): void
showVisitedLocations(): Iterator
<<Interface>>
EatingLodgingService
isFull(): boolean
addStudent(): void
removeStudent
givecurrentStudents(): TwoWayIterator
<<AbstractClass>>
ServiceClass
DEFAUT_INITIAL_RATING: final int
record: ServiceRecord
numberOfEvaluation: Map<Integer, Integer>
lastUpdated: int
averageRating: int
numberOfEvaluations: int
<<constructor>> ServiceClass(long log, long lat, int price, String name, int lastUpdated
<<AbstractClass>>
StudentClass
StudentRecord record
String home
String location
<<constructor>> ThriftyStudentClass(String name, String country, String home)
<<Record>>
StudentRecord
Name: String
type: String
Country: String
<<Class>>
EatingServiceClass
price: int
capacity: int
students: TwoWayList<Student>
<<constructor>> EatingServiceClass(long log, long lat, int price, int capacity, String name, int lastUpdated)
ServiceRecord
type: String
lat: long
log: long
name: String
<<Class>>
ThriftyStudentClass
cheapestEating: Service
<<construcotr>> ThriftyStudentClass(String name, String country, String home)
<<Class>>
BookishStudentClass
visitedLocations: Set<String, Service>
<<construcotr>> BookishStudentClass(String name, String country, String home)
hasVisited(String service): boolean
<<Class>>
LodgingServiceClass
price: int
capacity: int
students: TwoWayList<Student>
<<constructor>> LodgingServiceClass(long log, long lat, int price, int capacity, String name, int lastUpdated)
<<Class>>
LeisureServiceClass
price: int
<<constructor>> LeisureServiceClass(long log, long lat, int price, int discount, String name, int lastUpdated)
<<Class>>
OutgoingStudentClass
visitedLocations: Set<String, Service>
<<constructor>> OutgoingStudentClass(String name, String country, String home)
hasVisited(String service): boolean
<<package>>
DataStructure

- # Application
- Command *BOUNDS*:
    - Classes/Variáveis de Instância utilizadas:
        - HomeAwayAppClass - Bounds - String name
    - Complexidade temporal:
        - Melhor Caso e Pior Caso: O(1) - Visto que a única verificação depende do input dado pelo utilizador e do nome da aplicação, o número de passos será sempre constante
- Command *SAVE*:
    - Classes/Variáveis de Instância utilizadas:
        - HomeAwayAppClass - Bounds - String name
    - Complexidade Temporal:
        - Melhor e Pior Caso: O(n), n sendo o tamanho da classe principal - A serialização de todo o objeto depende do seu tamanho
            - Exemplo Melhor Caso: Salvar uma app que só possui as suas coordenadas e nome
            - Exemplo Pior Caso: Salvar uma app com milhares de serviços e estudantes
- Command *LOAD*:
    - Complexidade Temporal:
        - Melhor e Pior Caso: O(n), n sendo o tamanho da classe principal - É preciso ler todo o objeto e reconstruir todos os seus dados
            - Exemplo Melhor Caso: Ler uma app que só possui as suas coordenadas e nome
            - Exemplo Pior Caso: Ler uma app com milhares de serviços e estudantes
- Command *Service*:
    - Classes/Variáveis de Instância Utilizadas:
        - HomeAwayAppClass - serviceCollection, coordinates, numRatings (???)
        - Service Collection - servicesByName, servicesInInsertionOrder, ServicesOrderedByRating, ServicesByType, cheapestServiceByType, bestAverageByType
    - Complexidade Temporal:
        - Melhor Caso: O(log n), Pior Caso: O(n), n sendo o número total de serviços.
            - Exemplo melhor caso: O novo serviço é filho direto da root
            - Exemplo pior caso: Map servicesByName percorre todo o array para verificar se o serviço já existe
- Command Services:
    - Classes/Variáveis de Instância Utilizadas:
        - HomeAwayAppClass - serviceCollection
        - Services - ServicesRecord - name, lat, long, type
    - Complexidade Temporal:
        - Melhor Caso: O(1), Pior Caso: O(n), n sendo o número de serviços - visto que o iterador tem sempre de percorrer todos os serviços, o número de passos irá depender do número de serviços do sistema
            - Exemplo Melhor Caso: Existe um serviço no sistema
            - Exemplo Pior Caso: Existem milhares de serviços no sistema
- Command *Student*:
    - Classes/Variáveis de Instância Utilizadas:
        - HomeAwayAppClass - serviceCollection, studentsCollection
        - StudentsCollectionClass - studentsByName, studentsCountry, studentOrderedByName
    - Complexidade Temporal:
        - Melhor Caso: O(log n), Pior Caso: O(n + m), n sendo o número de estudantes -
            - Exemplo Melhor Caso: O novo estudante é filho direto da root
            - Exemplo Pior Caso: Map studentsByName percorre todo o array para verificar se o estudante já existe
- Command *Leave:*
    - Classes/Variáveis de Instância Utilizadas:
        - HomeAwayAppClass - studentsCollection, servicesCollection
        - EatingServiceClass/LodgingServiceClass - students
    - Complexidade Temporal:
        - Melhor Caso O(1), Pior Caso O(n + m), sendo n o número de estudantes e m o número de serviços - O método indexOf() são lineares ao tamanho da lista
            - Exemplo Melhor Caso: Tanto o estudante como o serviço são encontrados imediatamente nos mapas
            - Exemplo Pior Caso: Tanto o estudante como o serviço precisam de percorrer o array inteiro do map para serem encontrados
- Command *Students*:
    - Classes/Variáveis de Instância Utilizadas:
        - HomeAwayAppClass - studentsCollection
        - StudentsClass -location, StudentRecord - name(), type()
    - Complexidade Temporal:
        - Melhor Caso O(1), Pior Caso O(n), sendo n o número de estudantes - A devolução do iterador é sempre constante, o número de passos depende do número de estudantes do sistema
            - Exemplo Melhor Caso: Existe um estudante no sistema
            - Exemplo Pior Caso: Existem milhares de estudantes no sistema
- Command *Go:*
    - Classes/Variáveis de Instância Utilizadas:
        - HomeAwayAppClass - studentsCollection, servicesCollection
        - EatingServiceClass - students
        - StudentClass - location
        - ThriftyStudentClass - cheapestEating
    - Complexidade Temporal:
        - Melhor Caso O(1), Pior Caso O(n + m), sendo n o número de estudantes e m o número de serviços - Os métodos de search são geralmente constantes, assim como as verificações e alterações nos objetos
            - Exemplo Melhor Caso: Tanto o estudante como os serviços encontram-se imediatamente nos mapas e o estudante não é Thrifty
            - Exemplo Pior Caso: Tanto o estudante como os serviços precisam de percorrer o array inteiro do map para serem encontrados, o estudante é thrifty e o cheapestEating muda
- Command *Move*:
    - Classes/Variáveis de Instância Utilizadas:
        - HomeAwayAppClass - StudentsCollection, serviceCollection
        - LodgingServiceClass - students, capacity
        - StudentClass - home
    - Complexidade Temporal:
        - Melhor Caso O(1), Pior Caso O(n + m), sendo n o número de estudantes e m o número de serviços - Os métodos de search são geralmente constantes, assim como as verificações e alterações nos objetos
            - Exemplo Melhor Caso: Tanto o estudante como os serviços encontram-se imediatamente nos mapas

            - Exemplo Pior Caso:Tanto o estudante como os serviços precisam de percorrer o array inteiro do map para serem encontrados
- Command *Users*:
    - Classes/Variáveis de Instância Utilizadas:
        - HomeAwayAppClass - serviceCollection
        - EatingServiceClass/LodgingServiceClass - students
    - Complexidade Temporal:
        - Melhor Caso O(1), Pior Caso O(n + m), sendo o número de serviços e m o número de estudantes - como é necessário procurar pelo serviço, a complexidade temporal da mesma poderá ser de valor n (caso seja preciso percorrer o array inteiro). Ainda será necessário percorrer o iterador de estudantes desse serviço, logo a complexidade temporal do comando no pior caso será a soma das duas complexidades temporais
            - Exemplo Melhor Caso: Existe um estudante no serviço
            - Exemplo Pior Caso: O serviço está cheio
- Command *Where*:
    - Classes/Variáveis de Instância Utilizadas:
        - HomeAwayAppClass - studentsCollection
        - StudentsClass - location
    - Complexidade Temporal
        - Melhor Caso O(1), Pior Caso O(n), sendo n o número de estudantes - é necessário procurar o estudante para aceder à sua *location*
            - Exemplo Melhor Caso: O estudante encontra-se imediatamente no mapa
            - Exemplo Pior Caso: É preciso percorrer o array inteiro do mapa para encontrar o estudante
- Command *Visited*:
    - Classes/Variáveis de Instância Utilizadas:
        - HomeAwayAppClass - studentsCollection
        - ServiceCollectionClass - servicesOrderedByRating, servicesByType
        - BookishOutgoingStudent - visitedLocations
    - Complexidade Temporal
        - Melhor Caso O(1), Pior Caso O(n + m), sendo n o número de estudantes e m o número de locais visitados - É necessário procurar o estudante para aceder à sua lista de *visitedLocations*, e percorrer o iterador das suas visitas
            - Exemplo Melhor Caso: O estudante encontra-se imediatamente e visitou apenas um serviço
            - Exemplo Pior Caso: É preciso percorrer o array inteiro do mapa para encontrar o estudante, e o mesmo visitou todos os serviços do sistema
- Command *Star:*
    - Classes/Variáveis de Instância Utilizadas:
        - HomeAwayAppClass - serviceCollection, numRatings
        - ServiceClass - stars, averageRating, lastChanged, tags, numberOfEvaluations
    - Complexidade Temporal
        - Melhor Caso O(1), Pior Caso O(n), sendo n o número de serviços - É necessário localizar o serviço na coleção e, além disso, ao alterar a média da sua avaliação, o serviço pode ter de ser removido e reinserido na posição correta das estruturas de dados
            - Exemplo Melhor Caso: O serviço encontra-se imediatamente e a nova avaliação não altera a sua posição relativa no ranking
            - Exemplo Pior Caso: É preciso percorrer o array inteiro do mapa para encontrar o serviço e a alteração da sua média obriga a reordenar a lista de serviços por *ranking*
- Command *Ranking*:
    - Classes/Variáveis de Instância Utilizadas:
        - HomeAwayAppClass - serviceCollection
        - ServicesClass - averageRating, ServicesRecord - name()
    - Complexidade Temporal
        - Melhor Caso O(1), Pior Caso O(n), sendo n o número de serviços no sistema - Todos os métodos têm tempo constante, somente aumenta pois temos de iterar sobre todos os serviços
            - Exemplo Melhor Caso: Existe apenas um serviço no sistema
            - Exemplo Pior Caso: Existem milhares de serviços no sistema
- Command *Ranked*:
    - Classes/Variáveis de Instância Utilizadas:
        - HomeAwayAppClass - studentsCollection, serviceCollection
        - ServiceClass - averageRating
    - Complexidade Temporal
        - Melhor Caso O(1), Pior Caso O(n + m), sendo n o número de serviços e m o número de estudantes - Visto que temos de procurar o estudante e encontrar a sua localização, juntamente com iterar por todos os serviços, a pesquisa será linear, no pior dos casos
            - Exemplo Melhor Caso: O estudante encontra-se imediatamente e existem dois serviços, sendo um deles a localização do estudante
            - Exemplo Pior Caso: Tanto o estudante como a sua localização precisam de percorrer o array inteiro do map para serem encontrados e existem milhares de serviços para comparar, estando todos dentro das condições para serem devolvidos no iterador
- Command *Tags*:
    - Classes/Variáveis de Instância Utilizadas:
        - HomeAwayAppClass - serviceCollection
    - Complexidade Temporal
        - Melhor Caso O(1), Pior Caso O(n * (m + l)), sendo n o número de serviços no sistema, m o tamanho da tag do serviço e l o tamanho da tag - A devolução do iterador será sempre em tempo constante, aumenta os passos em função do número de serviços que terá de iterar
            - Exemplo Melhor Caso: Existe apenas um serviço no sistema com a tag
            - Exemplo Pior Caso: Existem milhares de serviços e nenhum deles possui a tag
- Command *Find:*
    - Classes/Variáveis de Instância Utilizadas:
        - HomeAwayAppClass - studentsCollection, serviceCollection
        - ServiceCollection - bestAverageByType, cheapestServiceByType
    - Complexidade Temporal
        - Melhor Caso O(1), Pior Caso O(n), sendo n o número de estudantes - Visto que os serviços mais relevantes de cada tipo já estão guardados na collection, este método só varia a partir da busca do estudante
            - Exemplo Melhor Caso: O estudante encontra-se imediatamente
            - Exemplo Pior Caso: É preciso percorrer o array inteiro do mapa para encontrar o estudante

Conclusion

Acreditamos que, com as estruturas de dados fornecidas para este trabalho, conseguimos um melhor desempenho no programa comparado do que a 1ª fase, sendo as complexidades menores que as do projeto anterior e mantendo um equilíbrio entre o desempenho e a adaptação para futuras mudanças